

MAC 233: Arduino Introduction

Dr Dan Brennan

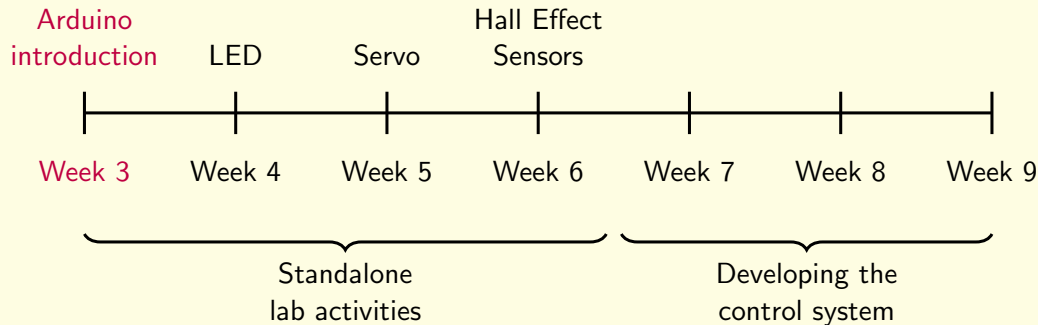
School of Mechanical, Aerospace, and Civil Engineering,
University of Sheffield, UK

MAC 233: **15/10/2025**



University of
Sheffield

MAC 233: LABS OVERVIEW

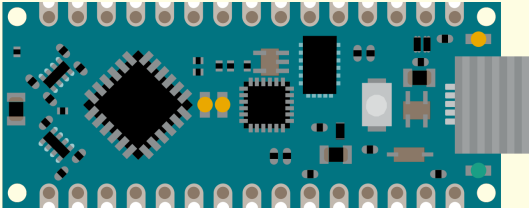


WHAT IS AN ARDUINO?

Arduino is an open-source electronics platform based on easy-to-use hardware and software. Arduino boards are able to read inputs - light on a sensor, a finger on a button, or a Twitter message - and turn it into an output - activating a motor, turning on an LED, publishing something online. You can tell your board what to do by sending a set of instructions to the microcontroller on the board. To do so you use the Arduino programming language (based on Wiring), and the Arduino Software (IDE), based on Processing.

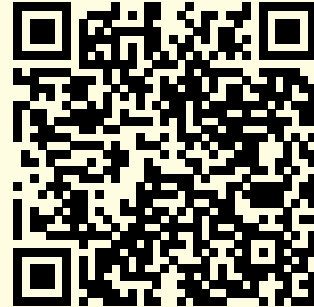
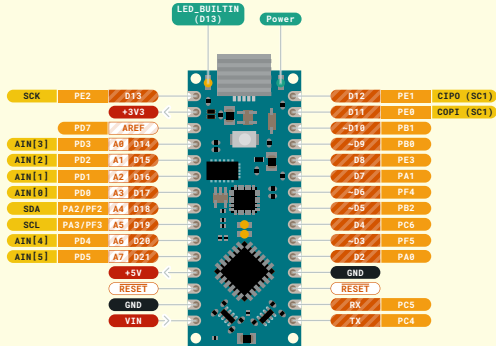
Source: <https://www.arduino.cc/en/Guide/Introduction>

ARDUINO NANO EVERY



Source: <https://docs.arduino.cc/hardware/nano-every/>

PINOUT DIAGRAM



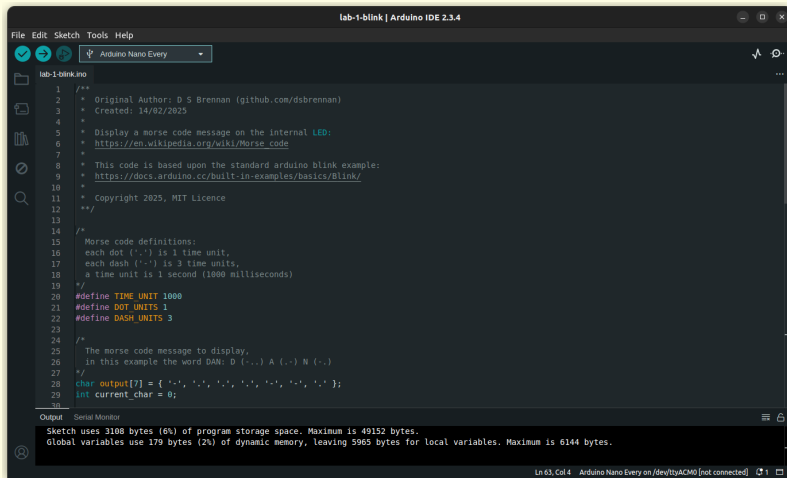
Source: <https://docs.arduino.cc/resources/pinouts/ABX00028-full-pinout.pdf>

MATLAB vs Arduino



University of
Sheffield

ARDUINO IDE



```
lab-1-blink.ino
1  /**
2   * Original Author: D S Brennan (github.com/dsbrennan)
3   * Created: 14/02/2025
4   *
5   * Display a morse code message on the internal LED:
6   * https://en.wikipedia.org/wiki/Morse\_code
7   *
8   * This code is based upon the standard arduino blink example:
9   * https://docs.arduino.cc/built-in-examples/basics/Blink/
10  *
11  * Copyright 2025, MIT Licence
12  **/
13
14  /**
15   * Morse code definitions:
16   * each dot ('.') is 1 time unit,
17   * each dash ('-') is 3 time units,
18   * a time unit is 1 second (1000 milliseconds)
19   */
20  #define TIME_UNIT 1000
21  #define DOT_UNITS 1
22  #define DASH_UNITS 3
23
24  /**
25   * The morse code message to display,
26   * in this example the word DAN: D (-..) A (..) N (..)
27   */
28  char output[7] = { '-', '.', '-', '.', '-', '.', '.' };
29  int current_char = 0;
30
```

Output Serial Monitor

Sketch uses 3108 bytes (6%) of program storage space. Maximum is 49152 bytes.
Global variables use 179 bytes (2%) of dynamic memory, leaving 5965 bytes for local variables. Maximum is 6144 bytes.

Ln 63, Col 4 Arduino Nano Every on /dev/ttyACM0 [not connected]

<https://www.arduino.cc/en/software>

MATLAB is a dynamically typed language.

This means you can simply declare a variable as below and MATLAB figures it out for you:

```
x = 10;
```

Arduino is a statically typed language.

This means when you declare a variable, you must let Arduino know what type of data is contained within the variable:

```
int x = 10;
```

VARIABLES: DATA TYPES

```
// integer: a whole number ranging from  
// -32,768 to 32,767  
int whole_number = 10;  
// long: a large whole number ranging from  
// -2,147,483,648 to 2,147,483,647  
long really_big_number = 3000000L  
// float: a decimal number ranging from  
// -3.4028235E+38 to 3.4028235E+38  
float decimal_number = 5.6;
```

VARIABLES: DATA TYPES

```
// unsigned: a value that doesn't  
// have a + or - in the value. This  
// allows the variable to store twice  
// the size e.g 0 to 65,535  
unsigned int whole_number = 10;  
// bool: a true or false  
bool correct = false;  
// array: a vector of char's (characters)  
char output[3] = { 'D', 'A', 'N'};
```

VARIABLES: QUALIFIERS

```
// marks the integer as a constant  
// and therefore is treated as a  
// read-only value  
const int SYSTEM_READY_LED_PIN = 5;
```

```
// marks the integer as volatile and keeps the  
// value in RAM. This must be done for any  
// variable that can be changed by an interrupt  
unsigned volatile int wheel_rotations_count;
```

A function is a grouping of logic into a named piece of code that you can then call from somewhere else in your code.

Arduino has two special functions that must exist in a Sketch before it can be compiled and uploaded to your Arduino: 'setup' and 'loop'.

MINIMUM SKETCH

```
void setup() {  
    // put your setup code here, to run once:  
}  
  
void loop() {  
    // put your main code here, to run repeatedly:  
}
```

The scope of a variable, is which areas of the program it is valid within e.g where can it be accessed.

A variable can either be within the scope of a function, or it can be within the scope of the whole program.

VARIABLES: SCOPE

```
// accessible from anywhere
```

```
int wheel_rotations = 0;
```

```
void setup() {
```

```
// accessible from within the setup function only
```

```
int setup_rotations = 1;
```

```
}
```

```
void loop() {
```

```
int loop_rotations = 2;
```

```
}
```

Control Structure



University of
Sheffield

CONTROL STRUCTURE: IF

You will often want to control what happens in your code depending on certain conditions.

The most useful control structure for our use case is an 'if'. If the condition is 'true', then the code within the curly brackets gets executed.

```
if (condition) {  
    // do something if condition is true  
}
```

CONTROL STRUCTURE: IF-ELSE

The 'if' control structure can be extended with an 'else'.

The code within the curly brackets after the 'else' gets executed if the condition is 'false'.

```
if (condition) {  
    // do something if condition is true  
} else {  
    // do something if condition is false  
}
```

CONTROL STRUCTURE: IF-ELSE-IF

This can be further extended to chain multiple conditions together.

```
if (condition1) {  
    // do something if condition1 is true  
} else if (condition2) {  
    // do something if condition1 is false  
    // but condition2 is true  
} else {  
    // do something if condition1 is false  
    // and condition2 is false  
}
```

CONDITION

A condition is the comparison of the 'left' value to the 'right value'.
A condition will be true if the comparison is correct.

' $X == Y$ ' The X and Y value are the same

' $X != Y$ ' The X and Y value are not the same

' $X < Y$ ' X is less than Y

' $X <= Y$ ' X is less than or equal to Y

Conditions can be chained together by either an and (&&) or an or (||).

CONDITION

```
int x = 10;
if (x > 21) {
    Serial.println("X is bigger than 21");
} else if (x >= 11 && x <= 20) {
    Serial.println("X is between 11 and 20");
} else if (x != 10) {
    Serial.println("X is less than 9");
} else {
    Serial.println("X is 10");
}
```

IO & Interrupts

Inputs & Outputs



University of
Sheffield

Inputs and Outputs from the Arduino are handled by the 'Pin's on the board.

Each Pin is numbered and can be configured as an input or an output. This decision must be made within the 'setup' function of your Sketch.

The 'pinMode' function sets the mode for a Pin. The first parameter is the pin number, and the second parameter is either 'INPUT' or 'OUTPUT'.

```
void setup() {  
    pinMode(3, INPUT);  
    pinMode(5, OUTPUT);  
}
```

PIN: SETTING THE OUTPUT VALUE

An output pin can either have the value set to LOW or HIGH.

```
void setup() {  
    // set pin 5 to output mode for an LED  
    pinMode(5, OUTPUT);  
    // turn 'off' the LED attached to pin 5  
    digitalWrite(5, LOW);  
    // turn 'on' the LED attached to pin 5  
    digitalWrite(5, HIGH);  
}
```

PIN: READING THE INPUT VALUE

An input could change at any point. As such, we can't rely upon the value to change when we are at a certain point in the 'loop' function.

Instead, we have to 'interrupt' the main 'loop' function to record the value.

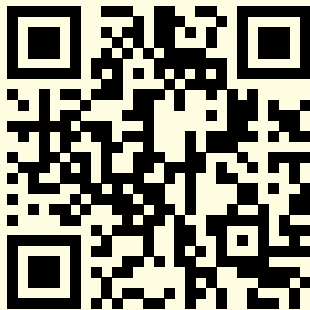
The 'interrupt' function can be called when the input value is 'LOW', 'CHANGE', 'RISING', or 'FALLING'.

PIN: READING THE INPUT VALUE

```
void setup() {  
    // set pin 3 to an input  
    pinMode(3, INPUT);  
    // when the input value of pin 3 falls  
    // call the function 'wheelInterrupt'  
    attachInterrupt(  
        digitalPinToInterrupt(3), wheelInterrupt, FALLING  
    );  
}  
  
void wheelInterrupt(){  
    // only called when the value associated  
    // with the pin has changed  
}
```

ADDITIONAL INFORMATION

I have only covered the most useful variables and structure for our use case; however, there are many more options available:



<https://docs.arduino.cc/language-reference>

Lab 1

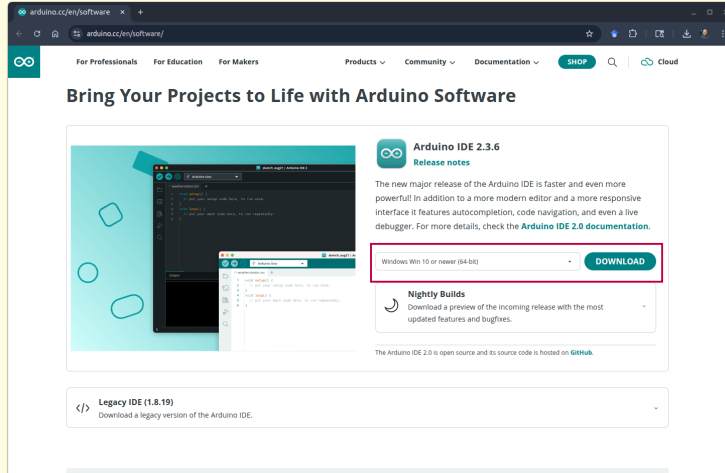
Communicating with the internal LED



University of
Sheffield

STEP 1: DOWNLOAD AND INSTALL ARDUINO IDE

Go to `https://www.arduino.cc/en/software`



STEP 2: DOWNLOAD LAB 1 FROM BLACKBOARD

Login to Blackboard: <https://vle.shef.ac.uk/>

Content / MAC233 Desi: x +

vle.shef.ac.uk/ultra/courses/_125430_1/outline

Student Preview

Content Calendar Announcements (1) Discussions (2) Gradebook Messages Groups

4(a) - Workshop - Arduino (Drs Brennan/Dolmansley)

- Workshop Labsheets
 - Lab 1: Morse code**
 - Lab 2: External LED
- Safe Use of the Take-Home/Self-Use Kits for Practical Teaching

You should read this document before attempting the Take Home Kit H+S Quiz This must be done before using take home kits unsupervised
- Risk Assessment for Take Home Kit Activities

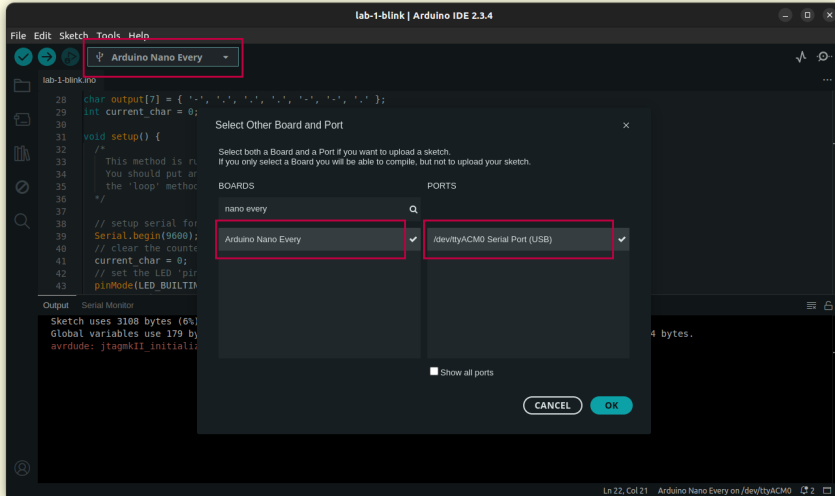
You will have to read and understand this document in order to pass the Health and Safety (H&S) quiz. If you have problems accessing this document, ensure you are logged into Google Drive via your sheffield.ac.uk account. Do not click "Request Access", as this will be ignored.
- Using Prototype Board (Breadboard)
- Kit - Component Data Sheets and Technical Documentation

STEP 2: DOWNLOAD LAB 1 FROM BLACKBOARD

Login to Blackboard: <https://vle.shef.ac.uk/>

The screenshot shows a web browser window with the URL vle.shef.ac.uk/ultra/courses/_125430_1/outline. The page is titled 'Student Preview' and shows a 'Contents' sidebar on the left. The sidebar lists various items, with 'Lab 1: Morse code' selected. The main content area is titled 'Lab 1: Morse code' and contains text about the lab's objective and the worksheet's purpose. At the bottom of the page, a file named 'lab-1-blink.pdf' is displayed, and a red box highlights the 'Download original file' button.

STEP 3: SELECT THE CORRECT BOARD AND PORT



Raise your hand and someone will come and help you.

